Blazor: sintassi Razor e data binding

Lezione 1 — 3IOT, SST-ICT-01 — Applicazioni web con Blazor

23/09/2026

1 Cos'è Blazor

Definizione

Blazor è un framework Microsoft per costruire pagine web interattive scrivendo il codice sia della parte visibile che della logica in **C#**, all'interno dello **stesso progetto**. Una pagina Blazor è formata da **due parti** che comunicano costantemente:

- il **front-end**: la parte **visibile** all'utente, scritta in **HTML** (con estensioni **Razor**).
- il **back-end**: la **logica applicativa**, scritta in C#, che genera i **dati** da mostrare nella pagina.

Nota

La connessione tra front-end e back-end viene stabilita all'apertura della pagina e resta **sempre aperta**: quando l'utente interagisce con la pagina (es. preme un **pulsante**), l'informazione arriva subito al back-end, che **elabora** e restituisce il risultato **senza dover ricaricare** la pagina intera.

1.1 Confronto con Windows Form e Razor Pages

Esempio

In una **Windows Form** (vista lo scorso anno), premere un pulsante avvia un **metodo** che viene eseguito e compie un'azione, tutto all'interno della stessa **applicazione desktop**.
Le pagine **Blazor** sfruttano concettualmente la stessa logica trasportata sul web: premere un pulsante nel front-end **chiama direttamente** un metodo scritto nel back-end.

Attenzione

Prima di Blazor esistevano le **Razor Pages**: lì non c'è una **connessione diretta** e continua tra front-end e back-end, ma un semplice **invio di dati**. Nelle Razor Pages, premere un pulsante chiama un metodo che genera una risposta, ma per aggiornare i dati visualizzati serve **ricaricare la pagina**. In Blazor questo non serve: i dati si aggiornano in **tempo reale**, **senza ricaricamento**.

2 SignalR: la tecnologia dietro la comunicazione in tempo reale

Definizione

SignalR è la tecnologia (una **libreria** .NET) che permette la **comunicazione diretta** e continua, in **tempo reale**, tra il front-end e il back-end di una pagina Blazor. "Real time" significa che i dati vengono scambiati man mano che cambiano, senza bisogno di ricaricare la pagina.

Nota

SignalR è nata originariamente per le **Razor Pages**, prima che questo insieme di tecnologie venisse riorganizzato e presentato sotto il nome "Blazor". Blazor è quindi un'**evoluzione** che integra SignalR in modo **trasparente**: usando Blazor, non serve preoccuparsi esplicitamente di come funziona SignalR sotto il cofano – è **già integrato** nel framework.

3 Creare un nuovo progetto Blazor

Definizione

In Visual Studio: ***Crea un nuovo progetto*** → ***App web Blazor***. Si sceglie un nome (es. `01_ripasso`) e la versione di `.NET` da usare (9 o 10, indifferentemente per gli scopi di questo corso).

Attenzione

Il progetto va **salvato** in una cartella **locale** (es. sul Desktop), **non** direttamente dentro una cartella sincronizzata con il cloud (Google Drive, OneDrive): l'***App Host***, il componente che avvia il **web server** dell'applicazione, non riesce a partire correttamente se il progetto si trova dentro una **cartella sincronizzata**. Il backup su cloud va fatto in un secondo momento, copiando la cartella già **completa e chiusa**.

4 Struttura del progetto

Definizione

Un progetto Blazor è organizzato in **cartelle**. Le due più rilevanti per iniziare sono dentro `Components`:

- **`Layout`**: contiene i **contenitori principali** della struttura grafica.
- **`Pages`**: contiene le **singole pagine** dell'applicazione (**multipagina**).

4.1 La cartella Layout

Definizione

Dentro `Layout` si trovano due file principali:

- **MainLayout**: rappresenta l'intera **struttura della pagina**. Il suo HTML definisce un **header** (la testata, in alto), un blocco che importa `NavMenu` (la barra laterale di navigazione), e un **`body`** centrale che conterrà il contenuto della pagina attiva.
- **NavMenu**: la barra laterale di **navigazione** (**sidebar**), con l'elenco dei link alle varie pagine del sito.

Esempio

Nel progetto di base, `NavMenu` contiene **tre link**: `Home`, **`Counter`** (un contatore dimostrativo) e **`Weather`** (una tabella di dati di esempio). Per aggiungere una **nuova pagina** al menu, basta copiare uno di questi blocchi di link, modificarlo e puntarlo alla nuova pagina.

4.2 La cartella Pages

Nota

Ogni pagina del sito è un **file separato** dentro **`Pages`**. Il progetto base include già `Home`, `Counter` e `Weather`, più due pagine speciali: una per gli errori di **pagina non trovata** e una per gli **errori di connessione** verso il back-end (es. se la connessione SignalR cade). Le pagine applicative aggiuntive vanno create qui.

5 Anatomia di una pagina Blazor: la pagina Counter

Definizione

Una pagina Blazor è un file **ibrido**: contiene una parte in **HTML** (il markup, cosa mostrare e come) e una parte in **C#**, racchiusa nel blocco **`@code { ... }`**.

Esempio

La pagina `Counter` di esempio:

```
<h1>Counter</h1>

<p role="status">Current count: @currentCount</p>

<button class="btn btn-primary" @onclick="IncrementCount">
    Click me
</button>

@code {
```

```
    private int currentCount = 0;

    private void IncrementCount() {
        currentCount++;
    }
}
```

Definizione

`button` è un normale **tag HTML**. Il parametro `@onclick` è la parte specifica di Blazor: collega il pulsante a un **metodo del back-end** (`IncrementCount`), che viene chiamato **a ogni click**. Il simbolo `@` davanti a un nome di **variabile** (es. `@currentCount`) inserisce nell'HTML il valore corrente di quella variabile: è il meccanismo di **data binding**.

Nota

Il metodo `IncrementCount` aumenta il valore della variabile `currentCount`. Grazie al data binding, la pagina non ha bisogno di essere **ricaricata**: il numero mostrato si **aggiorna automaticamente**, **senza refresh**, perché SignalR trasmette in tempo reale il **nuovo valore** dal back-end al front-end.

Attenzione

Questo comportamento è **molto diverso** da una **Razor Page classica**: lì, al click di un pulsante, il browser **invia** i dati al server, il server elabora e rimanda indietro **tutta la pagina ricaricata** con i nuovi dati. In Blazor, invece, **solo il singolo dato** cambiato viene aggiornato: il resto della pagina (menu, testata, altri elementi) **resta fermo**.

6 Bootstrap: lo stile grafico delle pagine

Definizione

L'aspetto grafico delle pagine è gestito da **Bootstrap**, un framework CSS molto diffuso che fornisce **classi pronte** per bottoni, colori, layout e altri elementi grafici.

Esempio

Le **classi dei pulsanti** Bootstrap più comuni: `btn-primary` (blu, standard), `btn-secondary` (grigio), oltre a varianti verde, rossa, gialla, celeste, bianca, nera, e un link semplice. Il nome esatto della classe si trova nella **documentazione ufficiale** di Bootstrap (sezione *Components* → *Buttons*), **consultabile liberamente** – anche durante una verifica o un esame.

7 Un metodo eseguito al caricamento della pagina

Definizione

Blazor mette a disposizione un **metodo speciale**, `OnInitializedAsync`, eseguito automaticamente **una sola volta**, al momento in cui la pagina viene caricata (non ripetutamente, a differenza del `loop` di un Arduino).

7.1 Esempio: un messaggio condizionale

Esempio

Aggiungere un **messaggio** che compare quando il **contatore** raggiunge 10:

```
<p>@messaggio</p>

@code {
    private int currentCount = 0;
    private string messaggio = "";

    private void IncrementCount() {
        currentCount++;
        AggiornaMessaggio();
    }

    private void AggiornaMessaggio() {
        if (currentCount >= 10) {
            messaggio = "Hai raggiunto 10!";
        } else {
            messaggio = "";
        }
    }
}
```

Nota

Scrivere il controllo `if` **una sola volta** dentro un **metodo dedicato** (`AggiornaMessaggio`), e richiamare quel metodo ogni volta che serve, è più corretto rispetto a **duplicare** lo stesso `if` in più punti del codice: se in futuro serve cambiare la **soglia** da 10 a un altro valore, la modifica va fatta in **un solo posto** invece che in tutti i punti dove il controllo era stato copiato.

Attenzione

L'`if` da solo, senza un `else` che azzeri il messaggio, farebbe restare il messaggio **visibile** anche se il contatore torna sotto soglia (es. con un pulsante di decremento aggiunto in seguito): va sempre previsto il caso **"altrimenti"**, non solo quello che attiva il messaggio.

8 L'autocompletamento IA in fase di apprendimento

Attenzione

Durante l'esercitazione, l'insegnante ha richiamato l'attenzione sull'uso dell'**autocompletamento basato su intelligenza artificiale** integrato in Visual Studio: se usato per scrivere il codice **al posto dello studente** (invece che come aiuto puntuale), **impedisce di imparare** a scrivere correttamente la **sintassi di base** (punto e virgola, parentesi, parole chiave come `void`). In una **verifica scritta** senza strumenti di completamento automatico, chi si è affidato troppo all'IA durante l'esercitazione rischia di **non saper scrivere** codice corretto **autonomamente**. L'insegnante ha annunciato che valuterà la disattivazione dei suggerimenti automatici durante le esercitazioni in classe.

9 Esercizio assegnato per casa

Esempio

Scrivere una **web app Blazor** che, nella pagina `Counter` (o una pagina analoga), mostri **10 pulsanti numerati** da 0 a 9 e un **pulsante di conferma**. In memoria è salvato un **codice a 4 cifre**, scelto liberamente dallo studente. Se l'utente inserisce la sequenza corretta di cifre (premendo i pulsanti numerati nell'ordine giusto) e conferma, deve comparire un messaggio di **"apertura"** – concettualmente una piccola simulazione di una **serratura elettronica** a codice.

Nota

Questo esercizio richiede di riutilizzare esattamente gli strumenti visti in questa lezione: più pulsanti con **`@onclick`** collegati a metodi diversi (o allo stesso metodo con un parametro), una **variabile** che accumula le cifre inserite, e un **confronto** (`if`) tra la **sequenza inserita** e il **codice memorizzato** per decidere se mostrare il messaggio di apertura.

10 Cosa ricordare di questa lezione

- **Blazor** collega front-end (HTML/Razor) e back-end (C#) con una **connessione continua**, grazie a **SignalR**: **niente ricaricamento** della pagina per aggiornare i dati.
- A differenza delle **Razor Pages** (invio dati + **ricaricamento completo**), Blazor aggiorna **solo i dati cambiati**.
- **Struttura** di un progetto: **`Layout`** (MainLayout, NavMenu) e **`Pages`** (le singole pagine).
- Una pagina Blazor è **ibrida**: markup HTML + blocco **`@code`** in C#. **`@onclick`** collega un elemento a un metodo; **`@variabile`** inserisce un valore nel markup (**data binding**).
- **`OnInitializedAsync`** è il metodo eseguito **una sola volta** al caricamento della pagina.
- **Bootstrap** fornisce le classi **CSS** per lo **stile grafico** (es. `btn-primary`).
- Il progetto Blazor va salvato **in locale**, non nel cloud, per non impedire l'avvio dell'App Host.